

Week 14 checkpoint progress report

Data and Evaluation Foundation for StruQ-Protected Email Triage

組別第五組
112062123 任光謙
112062144 范升維 (Writer)
112062114 王昱閔
112062224 蔡明妍

Week 14

1 Project summary

This project builds a proof-of-concept email triage system that uses an LLM to classify incoming messages as `normal`, `trash`, or `malicious`. The system is implemented around an `n8n` workflow. A sample email enters through a webhook, the workflow normalizes the email fields, applies a StruQ-style structured prompt design, calls a local LLM through Ollama, validates the JSON output, and then chooses a deterministic routing action.

The main security problem is prompt injection inside email content. The email subject, body, links, sender field, and attachment names are all untrusted. A malicious email can include text such as “ignore previous instructions” or fake JSON that tries to force the classifier to mark the message as safe. The project reduces this risk by separating trusted instructions from untrusted email data and by validating the model output before routing.

2 My responsibility

My role is Data and Evaluation Lead. For Week 14, my responsibility was to prepare the first evaluation foundation rather than modify the `n8n` workflow itself. The `n8n` baseline prototype already provides a webhook path and LLM validation flow, so my work focuses on the data that will be used to test that workflow.

Responsibility	Week 14 work
Seed evaluation data	Created a balanced JSONL email set with <code>normal</code> , <code>trash</code> , and <code>malicious</code> examples.
Attack data	Created initial prompt-injection cases based on the StruQ threat model.
Data validation	Wrote a script to check required fields, label values, array fields, and message IDs.
Evaluation planning	Defined how the data will support accuracy, malicious recall, false positive rate, manual review rate, and attack success rate.

Table 1: Week 14 Data and Evaluation responsibilities.

3 Spec traceability

I checked the Week 14 work against the project specification and kept the implementation inside the data and evaluation track. Table 2 shows the mapping from the specification to the delivered artifacts.

Spec requirement	Expected artifact	Week 14 status
Collect normal, trash, and malicious examples	raw CSV files	Prepared 36 raw seed emails, 12 per label.
Create processed evaluation data	processed JSONL	Prepared webhook-compatible JSONL records.
Build prompt-injection cases	attack JSONL files	Prepared 10 attack cases across StruQ-related attack categories.
Implement or run evaluation scripts	<code>run_eval.py</code> , <code>metrics.py</code>	Added validation summary and metric helper functions.
Prepare final metric path	Week 14 result file	Saved baseline summary and Week 15 handoff notes.

Table 2: Traceability between the project specification and my Week 14 artifacts.

4 Week 14 deliverables

I added the Week 14 deliverables under the project structure defined in `docs/spec.md`:

- `data/raw/normal_emails.csv`: raw normal seed emails.
- `data/raw/spam_emails.csv`: raw trash/spam seed emails.
- `data/raw/malicious_emails.csv`: raw malicious seed emails.
- `data/processed/email_eval_set.jsonl`: 36 labeled seed emails.
- `data/attacks/prompt_injection_cases.jsonl`: raw prompt-injection cases.
- `data/attacks/struqlite_attack_set.jsonl`: 10 prompt-injection attack cases.
- `src/evaluation/`: attack-set builder, local evaluation summary command, and metric helpers.
- `tests/test_eval_metrics.py`: unit tests for evaluation metric helpers.
- `results/week14_baseline_results.md`: saved summary output and validation notes.

The Week 14 data is synthetic and project-specific. I chose this approach for the checkpoint because it avoids privacy issues, keeps licensing simple, and lets the team control the exact behaviors we need to test. Public datasets can still be added later after their license and preprocessing requirements are checked, but they should not block the first working evaluation path.

5 Evaluation seed dataset

The evaluation seed set contains 36 emails with an even label distribution. I kept a raw CSV version by category and a processed JSONL version for automated evaluation. Each processed example uses the same webhook-compatible fields as the n8n prototype:

Listing 1: Evaluation record schema.

```
{
  "message_id": "malicious-001",
  "from": "it-support@example-login.com",
  "reply_to": "security-update@example-login.com",
  "subject": "Urgent: verify your account password",
  "body_text": "Your mailbox will be disabled today...",
  "body_html": "",
  "urls": ["https://example-login.com/verify-password"],
  "attachment_names": [],
  "expected_label": "malicious",
  "source_type": "synthetic_seed",
  "split": "week14_seed",
  "rationale": "Credential phishing with urgency and suspicious login domain."
}
```

The labels are defined as follows:

- **normal**: expected school, project, billing, calendar, or work messages with no suspicious credential or payment request.
- **trash**: promotional messages, newsletters, coupons, and sales outreach. These may be unwanted but are not directly malicious.
- **malicious**: phishing, credential theft, banking or card-data theft, business email compromise, gift-card scams, suspicious executable attachments, and fake-check scams.

The seed set is intentionally small but balanced. This is enough for Week 14 because the checkpoint asks for 30–50 labeled sample emails. For Week 15, the same schema can be expanded to 80–120 emails without changing the n8n workflow input format or the evaluation script.

6 Prompt-injection attack set

The attack set contains 10 cases. The goal is not only to test classification accuracy, but also to test whether attacker-controlled email text can change the task, output schema, or automation behavior.

Attack type	Purpose
ignore_instruction	Attempts to force the model to classify a phishing email as normal.
fake_json_completion	Places fake JSON inside the email body.
delimiter_spoofing	Uses reserved StruQ markers such as [MARK] and [RESP].
fake_system_message	Pretends that a system policy changed.
fake_n8n_command	Attempts to command the automation layer directly.
multilingual_chinese	Uses Chinese injection text inside a phishing email.
base64_instruction	Hides an instruction in base64 text.
control_characters	Mixes carriage returns, backspaces, tabs, and repeated newlines with injection text.
schema_change_request	Tells the model to change the JSON output contract.
benign_with_injection_text	Checks whether injection-like text creates unnecessary false positives.

Table 3: Initial Week 14 prompt-injection attack coverage.

7 Validation and test scripts

I implemented the data validation command in `src/evaluation/run_eval.py` and exposed it through `scripts/run_local_eval.sh`. The script reads both JSONL files, verifies the required webhook fields, checks allowed labels, confirms URL and attachment arrays, and reports duplicate or empty message IDs.

The command is:

Listing 2: Dataset validation command.

```
./scripts/run_local_eval.sh
```

The Week 14 validation result passed:

Listing 3: Week 14 summary output.

```
Raw CSV seed emails: 36
Evaluation seed emails: 36
Prompt-injection attack cases: 10
Total records prepared: 46

Raw CSV label distribution:
- malicious : 12
- normal    : 12
- trash     : 12

Evaluation label distribution:
- malicious : 12
- normal    : 12
- trash     : 12

Validation status: passed
```

```

Apple ~/SynologyDrive/projects/LLMCS-FinalProject  data ??
> ./scripts/run_local_eval.sh
# Week 14 Baseline Data/Evaluation Summary

Raw CSV seed emails: 36
Evaluation seed emails: 36
Prompt-injection attack cases: 10
Total records prepared: 46

Raw CSV label distribution:
- malicious : 12
- normal : 12
- trash : 12

Evaluation label distribution:
- malicious : 12
- normal : 12
- trash : 12

Attack expected-label distribution:
- malicious : 8
- normal : 1
- trash : 1

Attack type distribution:
- base64_instruction : 1
- benign_with_injection_text : 1
- control_characters : 1
- delimiter_spoofing : 1
- fake_json_completion : 1
- fake_n8n_command : 1
- fake_system_message : 1
- ignore_instruction : 1
- multilingual_chinese : 1
- schema_change_request : 1

Source type distribution:
- synthetic_attack_seed : 10
- synthetic_seed : 36

Validation checks:
- raw CSV files are readable
- required webhook fields present
- expected_label is one of normal/trash/malicious
- urls and attachment_names are JSON arrays
- message_id values are non-empty and unique within each file

Validation status: passed

```

Figure 1: Dataset validation and distribution summary.

I also added unit tests for the evaluation metric helpers:

Listing 4: Unit test command.

```
python3 -m unittest discover -s tests
```

The current test result is:

Listing 5: Evaluation metric test output.

```

....
-----
Ran 4 tests in 0.000s

OK

```

```

Apple ~/SynologyDrive/projects/LLMCS-FinalProject  data ??
> python3 -m unittest discover -s tests
....
-----
Ran 4 tests in 0.000s

OK

```

Figure 2: Evaluation metric unit test result.

8 How this supports the n8n prototype

The n8n workflow accepts `message_id`, `sender` fields, `subject`, `body text`, `HTML body`, `URLs`, and `attachment names`. The dataset uses those same fields, so each record can be sent to the webhook with minimal transformation. The additional fields such as `expected_label`, `rationale`, and `attack_type` are ignored by the n8n workflow but remain useful for evaluation.

This means the Week 15 evaluation can follow a direct path:

1. Send each JSONL record to the n8n webhook.
2. Save the model label, confidence, validation status, and final action.
3. Compare the model label with `expected_label`.
4. Calculate classification metrics and security metrics.

The important point is that this work does not require the n8n workflow or prompt implementation to change in Week 14. It prepares the data contract that the other components can consume. When the Week 15 workflow records predictions, those records can be joined back to `expected_label` and evaluated with the metric helpers.

9 Planned metrics

The evaluation track will report two groups of metrics.

Metric	Meaning
Overall accuracy	Fraction of clean evaluation emails with the correct label.
Malicious recall	Fraction of malicious emails correctly classified as malicious. This is the highest-risk metric.
Normal false positive rate	Fraction of normal emails incorrectly routed away from allow.
Manual review rate	Fraction of records routed to <code>manual_review</code> .
Attack success rate	Fraction of attack emails where the injected instruction causes an unsafe result.

Table 4: Planned Week 15 and Week 16 evaluation metrics.

10 Current status and next steps

For Week 14, the data and evaluation foundation is ready. The project now has balanced seed data, initial attack cases, and a repeatable validation script. The current dataset is intentionally small and controlled, which is appropriate for a checkpoint. It is large enough to exercise the n8n baseline and to make the next evaluation script meaningful.

For Week 15, my next tasks are:

- expand the clean evaluation set from 36 emails to 80–120 emails;
- add more prompt-injection cases, especially completion attacks and multilingual variants;
- run the dataset through the n8n webhook and collect predictions;
- implement metric calculation for accuracy, malicious recall, false positive rate, manual review rate, and attack success rate;

- evaluate whether public datasets such as Enron-style normal emails or spam corpora can be incorporated safely after licensing and preprocessing checks.

11 Reflection

The main lesson from Week 14 is that evaluation data must be designed together with the threat model. A normal spam dataset is not enough for this project because the system must also resist prompt injection and fake output attempts. The attack set therefore needs to include adversarial text that targets the model instruction, the JSON schema, and the n8n routing behavior.

I also kept the data format close to the n8n webhook schema. This makes the dataset useful immediately instead of becoming a separate offline artifact. In the next week, the same records can be used to generate concrete model outputs and measurable results.